

Migração para ORM

orm/ e routers_orm/ — as operações da Etapa 1 reescritas com SQLAlchemy

O que foi reimplementado

O item 4 pede todas as operações da Etapa 1 refeitas com uma ORM. As rotas em SQL puro continuam no ar, na raiz da API, e as versões com ORM ficam sob /orm. Manter as duas permite abrir a mesma tela apontando para uma ou para a outra e comparar o resultado.

Operação da Etapa 1	Rota em SQL puro	Rota com ORM
CRUD de paciente	/pacientes/	/orm/pacientes/
Preceptor	/preceptores/ (só criar e listar)	/orm/preceptores/ (CRUD completo)
CRUD de residente	/residentes/	/orm/residentes/
CRUD de unidade	/unidades/	/orm/unidades/
CRUD de procedimento	/procedimentos/	/orm/procedimentos/
Procedimento realizado	/procedimentos-realizados/	/orm/procedimentos-realizados/
CRUD de atendimento	/atendimentos/	/orm/atendimentos/
CRUD de escala	/escalas/	/orm/escalas/
Tempo médio por residente	/atendimentos/tempo-medio-por-residente	/orm/atendimentos/tempo-medio-por-residente
As 4 consultas analíticas	04_analiticas.sql, via psql	/orm/analiticas/...

As quatro consultas analíticas da Etapa 1 nunca tinham sido expostas pela API; rodavam pelo psql. Na Etapa 2 elas foram reescritas com a DSL e ganharam endpoint, porque "todas as operações" as inclui.

Por que SQLAlchemy

O SQLAlchemy já estava no projeto desde a Etapa 1, como gerenciador de conexão. Trocar de biblioteca significaria trocar o pool de conexões junto, sem ganho. Além disso ele é a opção recomendada no enunciado para Python, e tem uma característica útil aqui: a camada Core continua acessível, então a mesma sessão que manipula objetos mapeados também executa CALL e SELECT sobre view sem ginástica.

O engine é compartilhado. orm/sessao.py importa o engine de database.py em vez de criar outro, então as rotas da Etapa 1 e as da Etapa 2 disputam o mesmo pool de conexões, e não dois pools contra o mesmo banco.

Mapeamento das entidades

Treze classes em orm/modelos.py cobrem as onze tabelas da Etapa 1 e as duas da Etapa 2. Nada ali cria tabela: o esquema continua sendo responsabilidade dos scripts SQL, e o metadata é usado apenas para leitura. Rodar create_all() seria um segundo caminho para definir o banco, com risco de divergir dos scripts.

A herança não usa a herança da ORM

O DER tem duas especializações com naturezas diferentes. Pessoa para Paciente e Profissional é compartilhada e total: a mesma pessoa pode ser as duas coisas. Profissional para Preceptor e Residente é exclusiva.

A herança mapeada do SQLAlchemy, joined table inheritance, não representa o primeiro caso. Ela assume que uma linha da tabela pai corresponde a exatamente uma subclasse, e o mapa de identidade da sessão guarda um objeto por chave primária, de modo que a mesma pessoa não poderia ser carregada como Paciente e como Profissional ao mesmo tempo. O esquema também não tem coluna discriminadora, que a herança polimórfica usa para decidir qual classe instanciar.

As especializações foram mapeadas como relacionamentos um-para-um, com uselist=False. O SQL gerado é o mesmo da herança física, e a pessoa que for paciente e profissional continua representável.

orm/modelos.py

```
class Pessoa(Base):
    __tablename__ = "pessoa"
    id_pessoa: Mapped[int] = mapped_column(Integer, primary_key=True)
    CPF: Mapped[str] = mapped_column("cpf", String(11), unique=True, nullable=False)
    ...
    paciente: Mapped[Optional[Paciente]] = relationship(
        back_populates="pessoa", uselist=False, cascade="all, delete-orphan")
    profissional: Mapped[Optional[Profissional]] = relationship(
        back_populates="pessoa", uselist=False, cascade="all, delete-orphan")
```

CPF e CRM

O 01_schema.sql declara as colunas como CPF e CRM sem aspas, e o PostgreSQL guarda os dois nomes em minúsculo. Os schemas Pydantic da Etapa 1 esperam a forma maiúscula na resposta, o que na versão em SQL puro exigia escrever pe.CPF AS "CPF" em cada consulta. No mapeamento isso é resolvido uma única vez: o atributo se chama CPF e o primeiro argumento de mapped_column diz que a coluna real é cpf.

Sessão e transação

get_orm_db abre a sessão, entrega para a rota e fecha no finally. Não há commit automático: cada rota decide quando confirmar, e o close() descarta o que não foi confirmado.

A diferença mais visível em relação à Etapa 1 é o desaparecimento do try/except/rollback repetido em cada função. Um único auxiliar, routers_orm/comum.confirmar, faz o commit, captura o erro do banco, desfaz e traduz o SQLSTATE em código HTTP.

routers_orm/comum.py

```
CODIGOS_HTTP = {
    "23505": 409,      # unique_violation
    "23503": 400,      # foreign_key_violation
    "23502": 400,      # not_null_violation
    "23514": 400,      # check_violation
    "22023": 400,      # invalid_parameter_value
    "P0001": 409,      # raise_exception, das procedures e triggers
}
```

Na Etapa 1, qualquer falha virava 400 com a mensagem crua do psycopg2 no corpo da resposta. Agora o código HTTP corresponde ao tipo de problema e a mensagem sai limpa.

O mesmo cadastro nas duas camadas

Criar um preceptor envolve três tabelas. Em SQL puro são três INSERT em sequência, cada um repetindo o id devolvido pelo anterior:

routers/pessoa.py, Etapa 1

```
id_pessoa = db.execute(text("""
    INSERT INTO pessoa (nome, CPF, ...) VALUES (:nome, :cpf, ...)
    RETURNING id_pessoa;"""), {...}).scalar()

db.execute(text("""INSERT INTO profissional (id_pessoa, CRM, ...)
    VALUES (:id_pessoa, :crm, ...);"""), {"id_pessoa": id_pessoa, ...})

db.execute(text("""INSERT INTO preceptor (id_profissional, titulacao)
    VALUES (:id_pessoa, :titulacao);"""), {"id_pessoa": id_pessoa, ...})
db.commit()
```

Com ORM, os objetos são ligados e a sessão resolve a ordem:

routers_orm/pessoa.py, Etapa 2

```
pessoa = Pessoa(nome=dados.nome, CPF=dados.CPF, ...)
pessoa.profissional = Profissional(CRM=dados.CRM, ...)
pessoa.profissional.preceptor = Preceptor(titulacao=dados.titulacao)

db.add(pessoa)      # o cascade leva profissional e preceptor
confirmar(db)
```

db.add recebe apenas a pessoa. O cascade save-update alcança os objetos pendurados nela, e o grafo de dependências entre as chaves estrangeiras define a ordem dos INSERT. O id não aparece no código.

Chave primária composta

procedimento_realizado tem chave (id_atendimento, id_procedimento). O db.get recebe a tupla na ordem em que as colunas foram declaradas no modelo, o que substitui o WHERE com duas condições repetido em cada rota da Etapa 1.

routers_orm/procedimento_realizado.py

```
registro = db.get(ProcedimentoRealizado, (id_atendimento, id_procedimento))
```

Carregamento sob demanda e adiantado

O enunciado pede a demonstração de lazy loading contra eager loading. Por padrão o SQLAlchemy carrega relacionamento sob demanda: o objeto vem sem os relacionados, e o primeiro acesso a cada um dispara uma consulta. Percorrer uma lista lendo um campo do relacionado produz o problema N+1.

Estratégia	SQL gerado	Quando serve
padrão, sob demanda	uma consulta por relacionamento acessado	quando o relacionado quase nunca é lido
joinedload	LEFT JOIN na mesma consulta	lado "um" da relação, como atendimento.paciente
selectinload	segunda consulta com IN	coleções, evita multiplicar as linhas do JOIN
contains_eager	aproveita um JOIN que a consulta já tem	quando o filtro já obrigou a junção

As listagens usam `contains_eager`, porque o filtro por nome já exige a junção com pessoa. Aproveitar essa junção evita a segunda consulta que o `joinedload` faria.

`routers_orm/pessoa.py`

```
stmt = (select(Paciente)
        .join(Paciente.pessoa)
        .options(contains_eager(Paciente.pessoa),
                  selectinload(Paciente.alergias)))
if nome:
    stmt = stmt.where(Pessoa.nome.ilike(f"%{nome}%"))
```

A diferença é medida, não afirmada. `GET /etapa2/consultas/lazy-vs-eager` percorre os atendimentos das duas formas, conta as instruções emitidas com um ouvinte de evento do engine e confirma que o resultado é igual. Na interface, o botão fica na aba Consultas ORM.

Concorrência otimista no mapeamento

Escala declara `version_id_col`. A cada `UPDATE`, o SQLAlchemy acrescenta a versão lida ao `WHERE` e incrementa a coluna. Se nenhuma linha casar, o flush levanta `StaleDataError` em vez de sobrescrever alteração alheia. O detalhe está no documento sobre concorrência.

`orm/modelos.py`, classe `Escala`

```
versao: Mapped[int] = mapped_column(Integer, nullable=False,
                                     server_default=text("1"))
__mapper_args__ = {"version_id_col": versao}
```

Diferenças de comportamento em relação à Etapa 1

- Preceptor ganhou `PUT` e `DELETE`. A limitação da Etapa 1 vinha do custo de escrever mais dois blocos de SQL para três tabelas; com o mapeamento, as duas operações saíram quase de graça.
- Apagar paciente não apaga mais a pessoa quando ela também é profissional. A Etapa 1 removía de paciente e depois de pessoa, o que levaria embora o vínculo profissional da mesma pessoa. Agora a linha de pessoa só sai quando nenhum papel resta.
- A verificação de conflito de escala saiu da aplicação. A Etapa 1 fazia um `SELECT` antes do `INSERT`, o que deixa uma janela entre a consulta e a gravação. Agora quem recusa é a restrição do banco mais o trigger, e a resposta 409 vem da tradução do erro.
- Atendimento aceita `id_unidade`, e procedimento realizado aceita `data_hora_inicio`. Os campos novos ficaram em schemas separados, então as respostas da Etapa 1 continuam idênticas.
- As alergias do paciente saem em ordem alfabética. A Etapa 1 usa `string_agg` sem `ORDER BY`, e a ordem de "penicilina, latex" depende de como o banco leu as linhas. É a única diferença de conteúdo entre as duas camadas nas listagens.

Verificação

A aplicação expõe 53 rotas. A bateria de testes rodou contra um PostgreSQL 16 com os sete scripts aplicados, e comparou as duas camadas endpoint a endpoint: sete pares de listagens devolvem a mesma contagem, e `/pacientes/` devolve o mesmo conteúdo que `/orm/pacientes/`, com a ressalva da ordem das alergias. O relatório de tempo médio por residente sai idêntico nas duas.

A medição de carregamento confirmou a diferença entre as estratégias: percorrer os quinze atendimentos lendo o nome do paciente custa 11 consultas sob demanda e 1 consulta com joinedload, com o mesmo resultado.